

1. Conversor de Tempo de Missão

Este algoritmo converte um valor total em segundos para o formato de dias, horas, minutos e segundos.

- **Avaliação:** A lógica está correta e utiliza divisões inteiras e restos de divisão (módulo) para decompor o tempo. Note que no pseudocódigo, o símbolo $\div$ representa a divisão inteira e $\% \%$ o resto.

Python

None

```
def conversor_tempo():
    t = int(input("Tempo de missão (segundos): "))

    dias = t // 86400
    resto1 = t % 86400
    horas = resto1 // 3600
    resto2 = resto1 % 3600
    minutos = resto2 // 60
    segundos = resto2 % 60

    print(f"{dias} dias, {horas:02}:{minutos:02}:{segundos:02}")

conversor_tempo()
```

2. Equilíbrio de Ponte de Wheatstone e Circuitos

O algoritmo calcula a resistência desconhecida (R_x) em uma ponte de Wheatstone, a resistência total e a potência.

- **Avaliação:** Há uma inconsistência na fórmula da potência. O código indica $P = V / I$, mas a fórmula correta da potência elétrica é $P = V \cdot I$ ou $P = V^2 / R$. Além disso, a verificação de equilíbrio compara se $R_1 \cdot R_x = R_2 \cdot R_3$.

Python

None

```
def circuito_resistor():
    r1 = float(input("R1 ( $\Omega$ ): "))
    r2 = float(input("R2 ( $\Omega$ ): "))
    r3 = float(input("R3 ( $\Omega$ ): "))
    v_fonte = float(input("Fonte (V): "))
```

```

# Cálculo de Rx (Ponte de Wheatstone)
rx = (r3 * r2) / r1 # Ajustado conforme lógica de ponte
[cite: 11]

# Resistência Total (Associação Mista) [cite: 12]
r_total = ((r1 + rx) * (r2 + r3)) / (r1 + rx + r2 + r3)

i_total = v_fonte / r_total
p_total = v_fonte * i_total # Corrigido de V/I para V*I

# Verificação de Equilíbrio
equilibrado = (r1 * rx) == (r2 * r3)

print(f"Rx: {rx:.2f} Ω | R Total: {r_total:.2f} Ω")
print(f"I Total: {i_total:.2f} A | P Total: {p_total:.2f} W")
print(f"Equilibrado: {'Sim' if equilibrado else 'Não'}")

circuito_resistor()

```

3. Cálculo de Impostos e Preço Final

Calcula a cascata de impostos (IPI, ICMS e PIS/COFINS) sobre o preço de fábrica de um produto.

- **Avaliação:** O cálculo do IPI divide por 200, o que pode ser um erro específico do domínio ou erro de escrita (o padrão seria 100). A "base-pc" parece ser uma base de cálculo composta.

Python

```

None

def calculo_tributario():
    preco_fabrica = float(input("Preço Fábrica (R$): "))
    aliq_ipi = float(input("Alíquota IPI (%): "))
    aliq_icms = float(input("Alíquota ICMS (%): "))
    aliq_pis_confins = float(input("Alíquota PIS/COFINS (%): "))

    ipi = preco_fabrica * (aliq_ipi / 200) # Mantido /200
    conforme manuscrito
    base_icms = preco_fabrica + ipi [cite: 19]

```

```

    icms = base_icms * (aliq_icms / 100) [cite: 20]

    base_pc = (base_icms + icms) / (1 - (aliq_icms / 100)) #
Interpretando lógica
    pis_confins = base_pc * (aliq_pis_confins / 100) [cite: 25]

    preco_final = base_pc + pis_confins [cite: 26]
    carga_total = ((preco_final - preco_fabrica) / preco_fabrica)
* 100 [cite: 27]

    print(f"\nRelatório:\nPreço Final: R$ {preco_final:.2f}")
    print(f"Carga Tributária Total: {carga_total:.2f}%")

calculo_tributario()

```

4. Análise de Solo (SPT e Capacidade)

Classifica a qualidade do solo para fundações com base nos índices SPT e capacidade de carga.

- **Avaliação:** As condições utilizam operadores lógicos para definir se o solo é Inapto, Restrito ou Excelente.

Python

```

None
def analise_solo():
    spt = int(input("Índice SPT: "))
    cap_carga = float(input("Capacidade de Carga (kPa): "))

    if spt < 5 or cap_carga < 80: [cite: 34]
        print("INAPTO: Solo muito fraco - reforço necessário")
    elif spt < 10 or cap_carga < 150: [cite: 36]
        par = "SPT" if spt < 10 else "Cap. Carga"
        print(f"RESTRITO: Fundação especial recomendada (Limite:
{par})")
    elif spt >= 30 and cap_carga >= 300: [cite: 39, 40]
        print("EXCELENTE: Solo de alta capacidade")
    else:
        print("SITUAÇÃO NORMAL / NÃO ESPECIFICADA")

```

analise_solo()

5. Controle de Qualidade (Solda e Produção)

Dois algoritmos em um: validação de resistência de solda e monitoramento de velocidade de linha.

- Avaliação A (Solda): Utiliza uma tolerância de 8% para aprovação.
- Avaliação B (Velocidade): Verifica se a velocidade está acima do alvo ou abaixo de 90% do esperado (gargalo).

Python

None

```
def controle_qualidade():
    # Início A: Solda [cite: 43]
    res_nominal = float(input("Resistência Nominal (MPa): "))
    res_medida = float(input("Resistência Medida (MPa): "))

    tolerancia_abs = res_nominal * 0.08 [cite: 48]
    desvio = res_medida - res_nominal [cite: 49]

    if abs(desvio) <= tolerancia_abs: [cite: 50]
        print("SOLDA APROVADA")
    else:
        perc_desvio = (desvio / res_nominal) * 100
        print(f"SOLDA REPROVADA | Desvio: {desvio:+.2f} MPa
({perc_desvio:.2f}%)") [cite: 53]

    print("-" * 20)

    # Início B: Produção [cite: 55]
    vel_atual = float(input("Velocidade Atual (m/min): "))
    vel_alvo = float(input("Velocidade Alvo (m/min): "))

    if vel_atual > (vel_alvo * 1.10): # Interpretando o "1,10" do
manuscrito [cite: 58, 60]
        print("VELOCIDADE ALTA - Risco de posicionamento
incorreto")
    elif vel_atual < (vel_alvo * 0.90): [cite: 62]
        print("VELOCIDADE BAIXA - Gargalo na produção")
```

```
else:  
    print("VELOCIDADE OK")  
  
controle_qualidade()
```

Houve erros de sintaxe na execução dos códigos em Python e uma análise incoerente com o problema proposto.

NOTEBOOKLM

1. Conversão de Tempo de Missão

- **Interpretação:** O algoritmo recebe um tempo total em segundos e o decompõe em unidades maiores: dias, horas, minutos e segundos.
- **Avaliação:** A lógica é tecnicamente correta para conversão temporal, utilizando divisões inteiras (por 86400 para dias e 3600 para horas) e o operador de resto (%) para carregar o tempo remanescente para a próxima unidade.
- **Código Python:**

None

```
t = int(input("Tempo de missão (s): "))
dias = t // 86400
resto1 = t % 86400
horas = resto1 // 3600
resto2 = resto1 % 3600
minutos = resto2 // 60
segundos = resto2 % 60
print(f"{dias} dias, {horas:02d}:{minutos:02d}:{segundos:02d}")
```

2. Análise de Ponte de Resistores

- **Interpretação:** Calcula parâmetros de um circuito que remete a uma Ponte de Wheatstone, determinando uma resistência desconhecida (R_x), a resistência total, corrente e potência ¹.
- **Avaliação:** O algoritmo define corretamente a condição de equilíbrio ($R_1 \cdot R_x = R_2 \cdot R_3$). No entanto, há um **erro técnico** na fórmula da potência no pseudocódigo original ($P = V/I$); o correto na engenharia é $P = V \cdot I$ ou $P = V^2/R$ ¹.

None

```
r1 = float(input("R1 (Ohm): "))
r2 = float(input("R2 (Ohm): "))
r3 = float(input("R3 (Ohm): "))
v_fonte = float(input("Fonte (V): "))

rx = (r3 * r2) / r1
r_total = ((r1 + rx) * (r2 + r3)) / (r1 + rx + r2 + r3)
i_total = v_fonte / r_total
# Corrigindo a fórmula de potência para conformidade técnica (P = V
* I)
p_total = v_fonte * i_total

equilibrio = (r1 * rx) == (r2 * r3)
print(f"Rx: {rx:.2f}, Rtotal: {r_total:.2f}, Itotal: {i_total:.4f}")
```

```
print(f"Equilíbrio: {equilibrio}")
```

3. Cálculo de Carga Tributária e Preço Final

- Interpretação: Calcula o preço final de um produto industrializado somando impostos em cascata (IPI, ICMS e PIS/COFINS) sobre o preço de fábrica.
- Avaliação: O algoritmo reflete bem a estrutura tributária brasileira, onde o IPI compõe a base de cálculo do ICMS. É eficiente para gerar relatórios de transparência fiscal.
- Código Python:

None

```
preco_fabrica = float(input("Preço Fábrica (R$): "))
aliq_ipi = float(input("Alíquota IPI (%): "))
aliq_icms = float(input("Alíquota ICMS (%): "))
aliq_pis_cofins = float(input("Alíquota PIS/COFINS (%): "))

ipi = preco_fabrica * (aliq_ipi / 100)
base_icms = preco_fabrica + ipi
icms = base_icms * (aliq_icms / 100)
base_pc = base_icms + icms
pis_cofins = base_pc * (aliq_pis_cofins / 100)
preco_final = base_pc + pis_cofins
carga_total = ((preco_final - preco_fabrica) / preco_fabrica) * 100

print(f"Preço Final: R$ {preco_final:.2f}")
print(f"Carga Tributária Total: {carga_total:.2f}%")
```

4. Avaliação de Capacidade de Carga do Solo

- Interpretação: Classifica a viabilidade de um terreno para fundações com base no índice SPT e na capacidade de carga em kPa.
- Avaliação: Utiliza estruturas condicionais aninhadas de forma lógica, definindo três categorias: Inapto, Restrito e Excelente. É um bom exemplo de apoio à decisão em engenharia civil.
- Código Python:

None

```
spt = int(input("Índice SPT: "))
cap_carga = float(input("Capacidade de carga (kPa): "))

if spt < 5 or cap_carga < 80:
    print("INAPTO: solo muito fraco - reforço necessário")
elif spt < 10 or cap_carga < 150:
    print("RESTRITO: Fundação especial recomendada")
elif spt >= 30 and cap_carga >= 300:
```

```

        print("EXCELENTE: solo de alta capacidade")
    else:
        print("ANALISAR: Solo de média capacidade")

```

5. Controle de Qualidade de Soldagem (Início A)

- Interpretação: Valida a resistência de uma solda comparando a medida real com a nominal, aceitando um desvio de até 8%.
- Avaliação: O algoritmo é robusto pois calcula o desvio absoluto e relativo, fornecendo feedback detalhado em caso de reprovação (percentual e sinal do erro).
- Código Python:

None

```

res_nominal = float(input("Resistência Nominal (MPa): "))
res_medida = float(input("Resistência Medida (MPa): "))

tolerancia_abs = res_nominal * 0.08
desvio = res_medida - res_nominal

if -tolerancia_abs <= desvio <= tolerancia_abs:
    print("SOLDA APROVADA")
else:
    perc_desvio = (desvio / res_nominal) * 100
    print(f"SOLDA REPROVADA. Desvio: {desvio:+.2f} MPa ({perc_desvio:+.1f}%)")

```

6. Monitoramento de Velocidade de Produção (Início B)

- Interpretação: Compara a velocidade atual de uma linha com a meta, disparando alertas de risco por excesso ou gargalo por lentidão.
- Avaliação: Utiliza margens de tolerância de +10% e -10% para definir o status, sendo uma lógica clara para monitoramento industrial em tempo real.
- Código Python:

None

```

v_atual = float(input("Velocidade Atual (m/min): "))
v_alvo = float(input("Velocidade Alvo (m/min): "))

if v_atual > v_alvo * 1.10:
    print("VELOCIDADE ALTA - risco de posicionamento incorreto")
elif v_atual < v_alvo * 0.90:
    print("VELOCIDADE BAIXA - gargalo na produção")
else:
    print("VELOCIDADE OK")

```


Nenhum código em Python criado apresentou erro. As análises são todas consistentes com o problema proposto.

COMPARAÇÃO GEMINI X NOTEBOOKLM

Alguns códigos em Python não apresentaram resultados iguais entre as duas LLMs. O Gemini alterou uma fórmula proposta pelo Pseudocódigo (Problema 3), o que apresentou resultados diferentes.